LAPORAN UJIAN TENGAH SEMESTER
PEMROGRAMAN BERORIENTASI OBJEK

Analisis SDLC, Pemodelan UML, dan Implementasi Enkapsulasi Kotlin
Tema Studi Kasus: ITK-Print (Layanan Percetakan Mandiri)

Dosen Pengampu	Himawan Wicaksono, S.ST., M.T.
Penyusun	Fadilah Apriliya Sari
NIM	04221054
Program Studi	Teknik Elektro
Institusi	Institut Teknologi Kalimantan
Tahun Akademik	Semester Genap 2025/2026

Nama Mahasiswa	Fadilah Apriliya Sari
NIM	04221054
Mata Kuliah	Pemrograman Berorientasi Objek (TE2514027)
Dosen Pengampu	Himawan Wicaksono, S.ST., M.T.
Tema Studi Kasus	ITK-Print (Layanan Percetakan Mandiri)
Repository GitHub	https://github.com/April-08-04/UTS-Pemrograman-Berorientasi-Objek-ITK-Print
Tools yang Digunakan	Draw.io / Lucidchart, IntelliJ IDEA atau Kotlin Playground, Git, GitHub

Laporan disusun untuk memenuhi tugas Ujian Tengah Semester mata kuliah Pemrograman Berorientasi Objek. Studi kasus yang dipilih adalah ITK-Print, yaitu sistem percetakan mandiri sederhana yang menekankan keamanan data, validasi aturan bisnis, serta penerapan enkapsulasi dalam bahasa Kotlin.

BAB I
PENDAHULUAN

1.1 Latar Belakang

Sistem layanan percetakan mandiri di lingkungan kampus merupakan salah satu contoh sistem sederhana yang tetap relevan untuk penerapan konsep pemrograman berorientasi objek. Dalam sistem seperti ini terdapat beberapa komponen utama, yaitu pelanggan, dokumen, dan mesin cetak. Masing-masing komponen memiliki data dan tanggung jawab yang berbeda sehingga lebih tepat dimodelkan sebagai objek yang saling berinteraksi dalam suatu sistem.

Pada proses pencetakan dokumen, terdapat data-data sensitif yang harus dijaga dengan baik, seperti saldo pelanggan, jumlah tinta, dan ketersediaan kertas pada mesin. Jika data tersebut dikelola secara sembarangan, maka sistem dapat menghasilkan kondisi yang tidak valid, seperti pencetakan tanpa pembayaran, kertas menjadi negatif, atau penggunaan tinta tanpa kontrol yang jelas. Hal ini dapat menyebabkan kerugian serta ketidakkonsistenan data dalam sistem.

Oleh karena itu, studi kasus ITK-Print (Layanan Percetakan Mandiri) sangat cocok digunakan untuk menunjukkan manfaat penerapan paradigma pemrograman berorientasi objek, khususnya konsep enkapsulasi. Dengan enkapsulasi, data sensitif dapat dilindungi agar tidak dapat diakses atau diubah secara langsung dari luar class, serta memastikan bahwa setiap perubahan data dilakukan melalui mekanisme yang telah divalidasi. Selain itu, pendekatan OOP juga membantu dalam pengorganisasian kode sehingga sistem menjadi lebih terstruktur, modular, dan mudah dikembangkan.

1.2 Rumusan Masalah

Berdasarkan latar belakang pada Ujian Tengah Semester ini terdapat rumusan masalah yang terdiri dari:

1. Bagaimana merancang sistem ITK-Print dengan pendekatan berorientasi objek yang aman dan terstruktur?

2. Bagaimana memodelkan kebutuhan sistem menggunakan Use Case Diagram dan Class Diagram?

3. Bagaimana menerapkan enkapsulasi pada Kotlin agar data sensitif seperti saldo, tinta, dan kertas tidak dapat diubah sembarangan?

4. Bagaimana membuktikan bahwa program mampu menolak proses pencetakan yang melanggar aturan bisnis dan memproses pencetakan yang sah?

1.3 Tujuan

Berdasarkan rumusan masalah pada Ujian Tengah Semester ini terdapat tujuan yang terdiri dari:

1. Menganalisis kebutuhan sistem ITK-Print berdasarkan aturan bisnis yang diberikan.

2. Membuat desain sistem dalam bentuk Use Case Diagram dan Class Diagram.

3. Mengimplementasikan sistem menggunakan Kotlin berbasis console.

4. Menerapkan data hiding dan jalur resmi perubahan data melalui method public.

5. Melakukan simulasi untuk membuktikan bahwa sistem aman dan berjalan sesuai aturan bisnis.

1.4 Aturan Bisnis Studi Kasus

Tema yang digunakan pada laporan ini adalah ITK-Print (Layanan Percetakan Mandiri). Berdasarkan ketentuan soal UTS, entitas utama pada tema ini adalah Pelanggan, Dokumen, dan MesinPrint. Aturan bisnis yang berlaku adalah sebagai berikut:

1. Proses cetak gagal jika saldo pelanggan tidak mencukupi untuk membayar biaya cetak.

2. Proses cetak gagal jika tinta pada mesin habis.

3. Proses cetak gagal jika kertas pada mesin tidak mencukupi jumlah halaman dokumen.

4. Proses cetak berhasil jika seluruh kondisi terpenuhi, yaitu saldo cukup, tinta tersedia, dan kertas mencukupi.

BAB II
ANALISIS SDLC DAN MANAJEMEN PROYEK

2.1 Alasan Memilih Paradigma OOP

Sistem ITK-Print lebih aman dan lebih cocok dibangun menggunakan paradigma pemrograman berorientasi objek (Object-Oriented Programming) dibandingkan paradigma prosedural. Hal ini disebabkan karena sistem memiliki beberapa data sensitif yang harus dijaga dengan baik, seperti saldo pelanggan, jumlah tinta, dan jumlah kertas pada mesin cetak.

Dalam paradigma OOP, setiap data sensitif tersebut dapat disimpan di dalam class dan dilindungi menggunakan access modifier seperti private. Dengan demikian, data tidak dapat diakses atau diubah secara langsung dari luar class. Sebagai gantinya, perubahan data hanya dapat dilakukan melalui method public yang telah dilengkapi dengan validasi logika sesuai aturan bisnis.

Pendekatan OOP juga memungkinkan setiap objek memiliki tanggung jawab yang jelas. Objek Pelanggan bertanggung jawab terhadap data saldo, objek Dokumen menyimpan informasi jumlah halaman, dan objek MesinPrint bertanggung jawab terhadap pengelolaan tinta, kertas, serta proses pencetakan. Dengan pembagian tanggung jawab seperti ini, sistem menjadi lebih terstruktur, mudah dipahami, dan mudah dikembangkan.

Sebaliknya, jika paradigma prosedural digunakan dengan variabel global, maka data seperti saldo, tinta, dan kertas dapat diubah secara langsung oleh bagian program lain tanpa validasi. Hal ini berpotensi menimbulkan kesalahan fatal seperti pencetakan tanpa pembayaran, penggunaan kertas yang melebihi stok, atau tinta yang tetap digunakan meskipun sudah habis. Oleh karena itu, penggunaan paradigma OOP menjadi pilihan yang tepat untuk menjamin keamanan dan konsistensi data dalam sistem ITK-Print.

2.2 Contoh Masalah Fatal Jika Menggunakan Variabel Global

Jika sistem ITK-Print dibangun menggunakan pendekatan prosedural dengan variabel global, maka beberapa masalah fatal berikut dapat terjadi:1

1. Saldo pelanggan dapat dikurangi secara langsung tanpa validasi, sehingga memungkinkan terjadinya pencetakan meskipun saldo tidak mencukupi.

2. Sistem tetap dapat mencetak dokumen walaupun jumlah kertas tidak mencukupi, sehingga menyebabkan nilai kertas menjadi tidak realistis.
3. Tanpa validasi yang tepat, sistem dapat terus melakukan pencetakan walaupun tinta sudah habis, yang tidak sesuai dengan kondisi nyata.
4. Proses pencetakan dapat dijalankan tanpa pengecekan kondisi awal, sehingga melanggar aturan bisnis yang telah ditentukan.

Dengan demikian, OOP melalui konsep enkapsulasi lebih tepat digunakan untuk memastikan bahwa setiap perubahan data sensitif harus melalui mekanisme yang aman dan tervalidasi.

2.3 Perencanaan Pengerjaan dengan Git dan GitHub

Dalam pengerjaan proyek ini digunakan Git sebagai version control system dan GitHub sebagai media penyimpanan repository publik. Berdasarkan hasil implementasi pada repository GitHub, proses pengerjaan dilakukan secara bertahap sebagai berikut:

1. Membuat repository dengan nama: UTS-Pemrograman-Berorientasi-Objek-ITK-Print
2. Melakukan commit awal (Initial commit) untuk membuat struktur project
3. Menambahkan file penting ke dalam repository:
   - File program utama (Main.kt)
   - File laporan (.docx)
   - Diagram UML (Use Case & Class Diagram)
   - README.md sebagai dokumentasi
4. Melakukan beberapa commit secara bertahap, yaitu:
   - Initial commit
   - Add files via upload (menambahkan diagram & laporan)
   - Main.kt (menambahkan kode program)
   - README.md (menambahkan dokumentasi)
5. Semua perubahan disimpan langsung ke branch main tanpa proses pull request tambahan (karena pengerjaan individu)

Dengan pendekatan ini, seluruh proses pengembangan terdokumentasi dengan baik melalui riwayat commit di GitHub..

2.4 Rencana Commit

Berdasarkan riwayat commit pada repository, strategi commit yang dilakukan adalah:

1. Initial commit

   Membuat struktur awal repository
2. Add files via upload

   Menambahkan file laporan dan diagram UML
3. Main.kt

   Mengimplementasikan program utama sistem ITK-Print
4. README.md

   Menambahkan dokumentasi project di GitHub

   Strategi ini menunjukkan bahwa pengembangan dilakukan secara bertahap dan terstruktur.

2.5 Perintah Git yang Digunakan

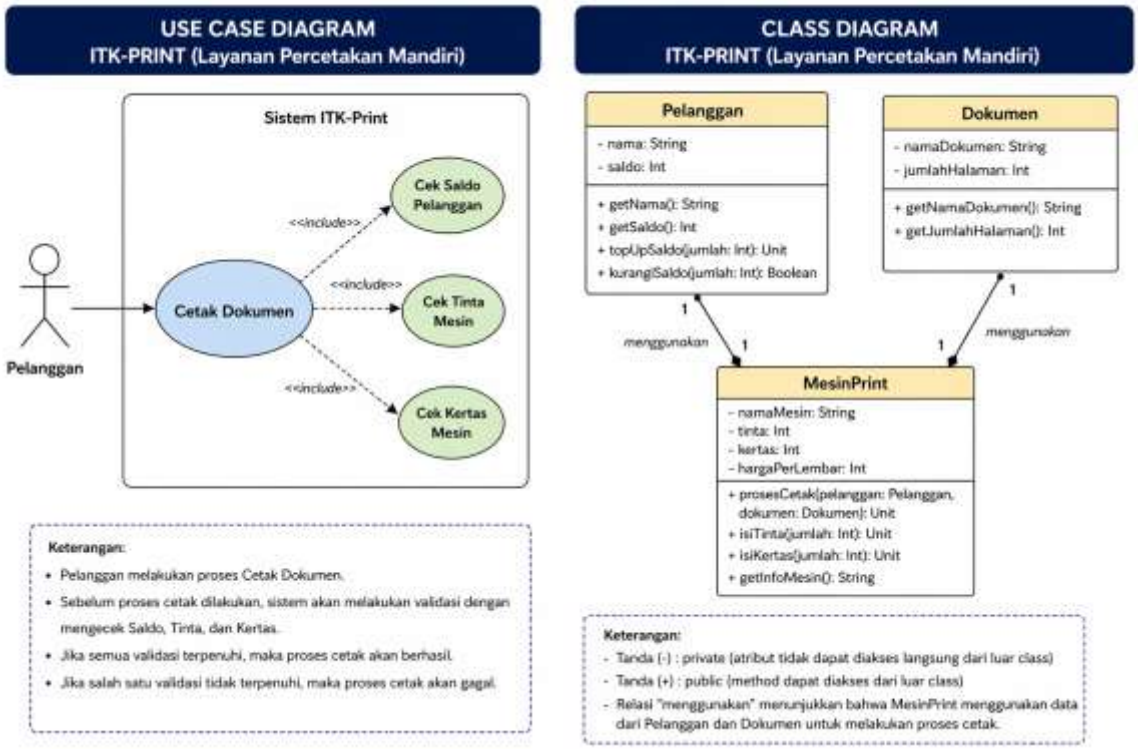
Berikut adalah perintah Git yang digunakan dalam pengerjaan proyek ini:

```
git init
git add .
git commit -m "Initial commit"

git add .
git commit -m "Add files via upload"

git add Main.kt
git commit -m "Menambahkan program utama ITK-Print"

git add README.md
git commit -m "Menambahkan dokumentasi project"

git push origin main
```

BAB III
PEMODELAN SISTEM

3.1 Use Case Diagram

Use Case Diagram digunakan untuk melihat sistem dari sudut pandang pengguna. Pada sistem ITK-Print terdapat satu aktor utama, yaitu Pelanggan. Aktivitas utama yang dilakukan adalah proses cetak dokumen. Sebelum proses cetak dijalankan, sistem wajib melakukan pengecekan saldo pelanggan, pengecekan tinta mesin, dan pengecekan kertas mesin. Oleh karena itu, ketiga proses tersebut dimodelkan menggunakan relasi <<include>> dari use case utama, sehingga proses cetak hanya dapat dilakukan apabila seluruh validasi terpenuhi.



Gambar 3.1 Use Case Diagram Sistem ITK-Print

Use Case Diagram digunakan untuk menggambarkan interaksi antara pengguna dengan sistem. Pada sistem ITK-Print, terdapat satu aktor utama yaitu Pelanggan.

1. Pelanggan berinteraksi dengan sistem untuk melakukan proses cetak dokumen.

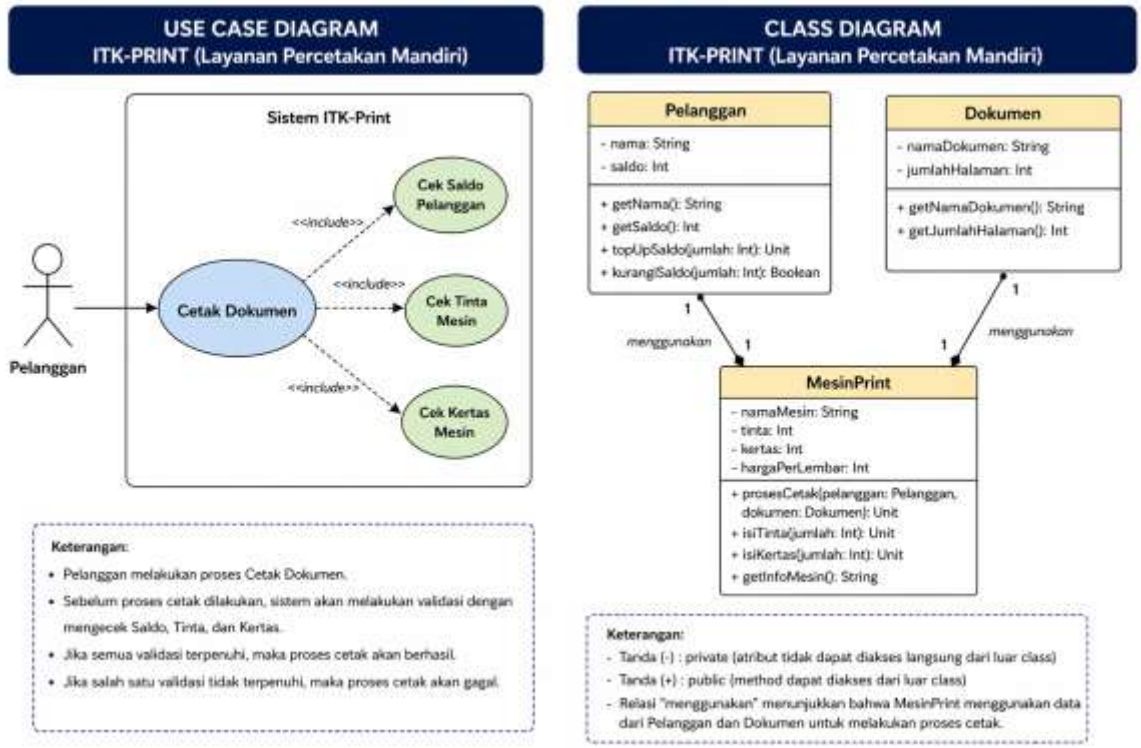
2. Sistem akan memproses permintaan cetak melalui use case utama yaitu Cetak Dokumen.
3. Use case Cetak Dokumen tidak dapat dijalankan secara langsung tanpa melalui beberapa proses validasi.
4. Sebelum proses cetak dilakukan, sistem wajib melakukan:
    - Cek Saldo Pelanggan
    - Cek Tinta Mesin
    - Cek Kertas Mesin
5. Ketiga proses tersebut dimodelkan menggunakan relasi <<include>>, yang berarti proses validasi selalu dijalankan sebelum proses cetak.
6. Jika semua kondisi terpenuhi, maka proses cetak akan berhasil. Sebaliknya, jika salah satu kondisi tidak terpenuhi, maka proses cetak akan gagal.

3.2 Class Diagram

Class Diagram merupakan gambaran struktur statis sistem yang menunjukkan hubungan antar class beserta atribut dan method yang dimiliki. Pada sistem ITK-Print, terdapat tiga class utama yaitu:

- Pelanggan
  Menyimpan data nama dan saldo pelanggan.
- Dokumen
  Menyimpan informasi dokumen seperti nama file dan jumlah halaman.
- MesinPrint
  Bertanggung jawab terhadap proses pencetakan, serta mengelola tinta, kertas, dan biaya per lembar.

Class MesinPrint memiliki peran utama dalam sistem karena menjadi penghubung antara data pelanggan dan dokumen dalam proses cetak. MesinPrint akan memanggil method yang ada pada class Pelanggan dan Dokumen untuk melakukan validasi sebelum proses pencetakan dilakukan.



Gambar 3.2 Class Diagram Sistem ITK-Print

3.3 Uraian Class dan Enkapsulasi

Pada class diagram, atribut seperti saldo pada Pelanggan, serta tinta dan kertas pada MesinPrint, diberi tanda minus (-) yang menunjukkan bahwa atribut tersebut bersifat *private*.

Hal ini sesuai dengan prinsip enkapsulasi dalam OOP, yaitu data sensitif tidak boleh diakses atau diubah secara langsung dari luar class. Sebagai gantinya, sistem menyediakan method public sebagai jalur resmi untuk memodifikasi data, yaitu:

- topUpSaldo() → untuk menambah saldo pelanggan
- kurangiSaldo() → untuk mengurangi saldo dengan validasi
- isiTinta() → untuk menambah tinta mesin
- isiKertas() → untuk menambah kertas mesin
- prosesCetak() → untuk menjalankan proses pencetakan dengan validasi lengkap

Di dalam method-method tersebut terdapat validasi logika (if-else) yang memastikan bahwa:

- Saldo pelanggan mencukupi
- Tinta mesin tersedia
- Kertas mesin mencukupi

Dengan desain ini, sistem menjadi lebih aman karena setiap perubahan data harus melalui mekanisme yang telah dikontrol. Selain itu, class diagram ini menjadi pedoman langsung dalam implementasi kode Kotlin, sehingga terdapat kesesuaian antara desain sistem dan hasil program.

.

BAB IV
IMPLEMENTASI PROGRAM KOTLIN

4.1 Pendahuluan

Bab ini membahas implementasi sistem **ITK-Print (Layanan Percetakan Mandiri)** ke dalam bentuk program menggunakan bahasa pemrograman Kotlin. Selain itu, pada bab ini juga disertakan proses pengujian serta pembahasan hasil implementasi untuk memastikan bahwa sistem telah berjalan sesuai dengan perancangan dan aturan bisnis yang telah ditentukan.

Implementasi dilakukan menggunakan pendekatan **Object-Oriented Programming (OOP)** dengan tujuan menghasilkan sistem yang terstruktur, aman, dan mudah dikembangkan. Pengujian dilakukan untuk memverifikasi bahwa seluruh fitur sistem berfungsi dengan baik serta mampu menangani kondisi gagal maupun sukses.

4.2 Gambaran Umum Sistem

Sistem ITK-Print dirancang untuk mensimulasikan layanan percetakan mandiri. Sistem ini melibatkan tiga komponen utama, yaitu:

- **Pelanggan** → menyimpan data pengguna dan saldo
- **Dokumen** → menyimpan data dokumen yang akan dicetak
- **MesinPrint** → memproses pencetakan dan validasi sistem

Setiap komponen diimplementasikan dalam bentuk class yang saling berinteraksi dalam proses pencetakan dokumen.

4.3 Struktur dan Implementasi Program

Program diimplementasikan dalam satu file utama (Main.kt) yang terdiri dari tiga class utama dan satu fungsi main().

4.3.1 Class Pelanggan

Class ini bertanggung jawab dalam pengelolaan data pelanggan, khususnya saldo. Atribut saldo bersifat private sehingga tidak dapat diakses langsung dari luar class.

Method yang disediakan:

- getNama()

- getSaldo()
- topUpSaldo()
- kurangiSaldo()

Validasi dilakukan pada method kurangiSaldo() untuk memastikan saldo tidak menjadi negatif.

4.3.2 Class Dokumen

Class Dokumen digunakan untuk menyimpan informasi dokumen yang akan dicetak, yaitu:

- nama dokumen
- jumlah halaman

Class ini hanya menyediakan method getter karena data bersifat tetap selama proses.

4.3.3 Class MesinPrint

Class ini merupakan inti sistem. MesinPrint bertugas:

- menghitung biaya cetak
- memvalidasi kondisi sistem
- menjalankan proses pencetakan

Validasi yang dilakukan meliputi:

- jumlah halaman valid
- tinta tersedia dan cukup
- kertas tersedia dan cukup
- saldo pelanggan mencukupi

Jika semua kondisi terpenuhi, maka proses pencetakan akan dilakukan dan data akan diperbarui.

4.4 Penerapan Enkapsulasi dan Validasi

Sistem menerapkan konsep **enkapsulasi** dengan menjadikan atribut sebagai private. Hal ini bertujuan untuk melindungi data dari perubahan langsung. Perubahan data hanya dapat dilakukan melalui method public yang telah dilengkapi validasi, seperti:

- topUpSaldo()
- kurangiSaldo()
- isiTinta()

- isiKertas()
- prosesCetak()

Validasi dilakukan menggunakan kondisi if-else sehingga sistem dapat menolak kondisi yang tidak sesuai aturan.

4.5 Pengujian Sistem

Pengujian dilakukan untuk memastikan bahwa sistem bekerja sesuai dengan aturan bisnis. Metode yang digunakan adalah **black-box testing**, yaitu pengujian berdasarkan input dan output.

4.5.1 Skenario Pengujian Gagal

Kondisi:

- Saldo pelanggan = Rp1000
- Total biaya = Rp5000

Ekspektasi:

Sistem menolak pencetakan.

Hasil:

Cetak gagal: saldo pelanggan tidak mencukupi.

```
=== SIMULASI GAGAL: SALDO TIDAK CUKUP ===

=== PROSES CETAK ===
Mesin      : Printer ITK-01
Pelanggan  : Fadilah
Dokumen    : Laporan_Gagal.pdf
Halaman    : 10
Biaya      : Rp5000
Cetak gagal: saldo pelanggan tidak mencukupi.
```

Analisis:

Sistem berhasil mendeteksi bahwa saldo tidak mencukupi, sehingga proses dihentikan.

4.5.2 Skenario Pengujian Sukses

Kondisi:

- Saldo pelanggan = Rp10000

Total biaya = Rp5000

Ekspektasi:

Sistem mencetak dokumen.

Hasil:

Cetak berhasil.

Saldo pelanggan tersisa: Rp5000

Sisa kertas mesin: 40

Sisa tinta mesin: 40

Analisis:

Sistem berhasil menjalankan proses cetak dan memperbarui data sesuai aturan.

```
=== PROSES CETAK ===
Mesin      : Printer ITK-01
Pelanggan  : Fadilah
Dokumen    : Laporan_Sukses.pdf
Halaman    : 10
Biaya      : Rp5000
Cetak berhasil.
Saldo pelanggan tersisa: Rp5000
Sisa kertas mesin: 40
Sisa tinta mesin : 40
```

4.6 Hasil Output Program

=== SISTEM ITK PRINT BERJALAN ===

=== SIMULASI GAGAL: SALDO TIDAK CUKUP ===

=== PROSES CETAK ===

Mesin : Printer ITK-01

Pelanggan : Fadilah

Dokumen : Laporan_Gagal.pdf

Halaman : 10

Biaya : Rp5000

Cetak gagal: saldo pelanggan tidak mencukupi.

=== SIMULASI SUKSES ===

=== PROSES CETAK ===

Mesin : Printer ITK-01

Pelanggan : Fadilah

Dokumen : Laporan_Sukses.pdf

Halaman : 10

Biaya : Rp5000

Cetak berhasil.

Saldo pelanggan tersisa: Rp5000

Sisa kertas mesin: 40

Sisa tinta mesin : 40

=== INFO AKHIR MESIN ===

Mesin: Printer ITK-01 | Tinta: 40 | Kertas: 40 | Harga/Lembar: Rp500

4.7 Pembahasan

Berdasarkan hasil pengujian, sistem mampu membedakan kondisi valid dan tidak valid dengan baik. Validasi dilakukan secara berurutan sehingga sistem dapat menghentikan proses lebih awal jika ditemukan kesalahan. Selain itu, penerapan OOP dan enkapsulasi terbukti efektif dalam menjaga keamanan data serta meningkatkan struktur program.

4.8 Kesimpulan Implementasi

Implementasi sistem ITK-Print telah berhasil dilakukan sesuai dengan desain. Sistem mampu menjalankan proses pencetakan dengan benar serta menjaga konsistensi data melalui validasi yang ketat.

BAB V
KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan hasil implementasi dan pengujian, sistem ITK-Print berhasil dibangun menggunakan paradigma **Object-Oriented Programming**. Sistem mampu:

1. Mengelola data pelanggan, dokumen, dan mesin
2. Menerapkan enkapsulasi untuk melindungi data
3. Melakukan validasi sebelum proses cetak
4. Menangani kondisi gagal dan sukses dengan benar

Dengan demikian, sistem telah memenuhi seluruh kriteria yang ditentukan dalam tugas uts.

5.2 Saran

Untuk pengembangan lebih lanjut, sistem dapat ditingkatkan dengan:

1. Menambahkan antarmuka berbasis GUI
2. Menyimpan data menggunakan database
3. Mendukung multi-user
4. Menambahkan fitur riwayat transaksi

Lampiran

```
class Pelanggan(
    private val nama: String,
    saldoAwal: Int
) {
    private var saldo: Int = saldoAwal

    fun getNama(): String {
        return nama
    }

    fun getSaldo(): Int {
        return saldo
    }

    fun topUpSaldo(jumlah: Int) {
        if (jumlah > 0) {
            saldo += jumlah
            println("Top up berhasil. Saldo ${nama} sekarang: Rp$saldo")
        } else {
            println("Top up gagal. Jumlah top up harus lebih dari 0.")
        }
    }

    fun kurangiSaldo(jumlah: Int): Boolean {
        return if (jumlah > 0 && saldo >= jumlah) {
```

```
            saldo -= jumlah
            true
        } else {
            false
        }
    }
}

class Dokumen(
    private val namaDokumen: String,
    private val jumlahHalaman: Int
) {
    fun getNamaDokumen(): String {
        return namaDokumen
    }

    fun getJumlahHalaman(): Int {
        return jumlahHalaman
    }
}

class MesinPrint(
    private val namaMesin: String,
    tintaAwal: Int,
    kertasAwal: Int,
    private val hargaPerLembar: Int
) {
```

```
    private var tinta: Int = tintaAwal
    private var kertas: Int = kertasAwal

    fun getInfoMesin(): String {
        return "Mesin: $namaMesin | Tinta: $tinta | Kertas: $kertas | Harga/Lembar:
Rp$hargaPerLembar"
    }

    fun isiTinta(jumlah: Int) {
        if (jumlah > 0) {
            tinta += jumlah
            println("Tinta berhasil ditambah $jumlah. Total tinta: $tinta")
        } else {
            println("Isi tinta gagal. Jumlah harus lebih dari 0.")
        }
    }

    fun isiKertas(jumlah: Int) {
        if (jumlah > 0) {
            kertas += jumlah
            println("Kertas berhasil ditambah $jumlah. Total kertas: $kertas")
        } else {
            println("Isi kertas gagal. Jumlah harus lebih dari 0.")
        }
    }

    fun prosesCetak(pelanggan: Pelanggan, dokumen: Dokumen) {
```

```
val halaman = dokumen.getJumlahHalaman()
val totalBiaya = halaman * hargaPerLembar

println("\n=== PROSES CETAK ===")
println("Mesin      : $namaMesin")
println("Pelanggan  : ${pelanggan.getNama()}")
println("Dokumen    : ${dokumen.getNamaDokumen()}")
println("Halaman    : $halaman")
println("Biaya      : Rp$totalBiaya")

if (halaman <= 0) {
    println("Cetak gagal: jumlah halaman dokumen tidak valid.")
    return
}

if (tinta <= 0) {
    println("Cetak gagal: tinta mesin habis.")
    return
}

if (kertas <= 0) {
    println("Cetak gagal: kertas mesin habis.")
    return
}

if (kertas < halaman) {
```

```
            println("Cetak gagal: kertas tidak mencukupi untuk mencetak $halaman
halaman.")
            return
        }

        if (tinta < halaman) {
            println("Cetak gagal: tinta tidak mencukupi untuk mencetak $halaman
halaman.")
            return
        }

        if (!pelanggan.kurangiSaldo(totalBiaya)) {
            println("Cetak gagal: saldo pelanggan tidak mencukupi.")
            return
        }

        kertas -= halaman
        tinta -= halaman

        println("Cetak berhasil.")
        println("Saldo pelanggan tersisa: Rp${pelanggan.getSaldo()}")
        println("Sisa kertas mesin: $kertas")
        println("Sisa tinta mesin : $tinta")
    }
}

fun main() {
```

```
    println("=== SISTEM ITK PRINT BERJALAN ===")

    // Simulasi gagal: saldo tidak cukup
    val pelangganGagal = Pelanggan("Fadilah", 1000)
    val dokumenGagal = Dokumen("Laporan_Gagal.pdf", 10)
    val mesinGagal = MesinPrint("Printer ITK-01", tintaAwal = 50, kertasAwal =
50, hargaPerLembar = 500)

    println("\n=== SIMULASI GAGAL: SALDO TIDAK CUKUP ===")
    mesinGagal.prosesCetak(pelangganGagal, dokumenGagal)

    // Simulasi sukses
    val pelangganSukses = Pelanggan("Fadilah", 10000)
    val dokumenSukses = Dokumen("Laporan_Sukses.pdf", 10)
    val mesinSukses = MesinPrint("Printer ITK-01", tintaAwal = 50, kertasAwal
= 50, hargaPerLembar = 500)

    println("\n=== SIMULASI SUKSES ===")
    mesinSukses.prosesCetak(pelangganSukses, dokumenSukses)

    println("\n=== INFO AKHIR MESIN ===")
    println(mesinSukses.getInfoMesin())
}
```

Kotlin

Solutions | Case studies | Docs | API | Community | Teach | Play

2.3.21 | JVM | Program arguments | Copy link | Share code | Run

```kotlin
    mesinGagal.prosesCetak(pelangganGagal, dokumenGagal)

    // Simulasi sukses
    val pelangganSukses = Pelanggan("Fadilah", 10000)
    val dokumenSukses = Dokumen("Laporan_Sukses.pdf", 10)
    val mesinSukses = MesinPrint("Printer ITK-01", tintaAwal = 50, kertasAwal = 50, hargaPerLembar = 500)

    println("\n=== SIMULASI SUKSES ===")
    mesinSukses.prosesCetak(pelangganSukses, dokumenSukses)

    println("\n=== INFO AKHIR MESIN ===")
    println(mesinSukses.getInfoMesin())
}
```

```
=== SISTEM ITK PRINT BERJALAN ===

=== SIMULASI GAGAL: SALDO TIDAK CUKUP ===

=== PROSES CETAK ===
Mesin      : Printer ITK-01
Pelanggan  : Fadilah
Dokumen    : Laporan_Gagal.pdf
Halaman    : 10
Biaya      : Rp5000
```

UTS-Pemrograman-Berorientasi-Objek-ITK-Print

UTS Pemrograman Berorientasi Objek

Sistem ITK-Print (Kotlin)

Commits